

Tutorial: Prognose des Bedarfs eines Fahrradverleihs mit Zeitreihenanalyse und ML.NET

Erfahren Sie, wie Sie den Bedarf für einen Fahrradverleih mithilfe einer univariaten Zeitreihenanalyse für in einer SQL Server-Datenbank gespeicherte Daten mit ML.NET prognostizieren können.

In diesem Tutorial lernen Sie, wie die folgenden Aufgaben ausgeführt werden:

- ✓ Das Problem verstehen
- ✓ Laden von Daten aus einer Datenbank
- ✓ Erstellen eines Vorhersagemodells
- ✓ Auswerten eines Vorhersagemodells
- ✓ Speichern eines Vorhersagemodells
- ✓ Verwenden eines Vorhersagemodells

Voraussetzungen

- [Visual Studio 2022](#) mit installierter Workload „.NET-Desktopentwicklung“.

Übersicht über das Beispiel zur Zeitreihenvorhersage

Bei diesem Beispiel handelt es sich um eine **C#.NET Core-Konsolenanwendung**, die den Bedarf für einen Fahrradverleih mithilfe eines univariaten Zeitreihenanalyse-Algorithmus prognostiziert, der unter dem Namen „singuläre Spektralanalyse“ bekannt ist. Den Code für dieses Beispiel finden Sie im Repository [dotnet/machinelearning-samples](#) auf GitHub.

Das Problem verstehen

Um einen effizienten Betrieb zu gewährleisten, spielt die Inventarverwaltung eine wichtige Rolle. Zu viel von einem Produkt auf Lager zu haben, bedeutet, dass unverkaufte Produkte in den Regalen stehen und keine Umsätze generieren. Zu wenig Produktvorrat führt zu Umsatzeinbußen und Kunden, die von Wettbewerbern kaufen. Daher stellt sich immer wieder die folgende Frage: Wie hoch ist die optimale Menge an Lagerbestand, die vorgehalten werden sollte? Die Zeitreihenanalyse hilft, eine Antwort auf diese Fragen zu geben, indem sie historische Daten untersucht, Muster identifiziert und diese Informationen

verwendet, um Werte irgendwann in der Zukunft vorherzusagen.

Das Verfahren zum Analysieren von Daten, das in diesem Tutorial verwendet wird, ist die univariate Zeitreihenanalyse. Die univariate Zeitreihenanalyse untersucht eine einzelne numerische Beobachtung über einen bestimmten Zeitraum in bestimmten Intervallen, z.B. die monatlichen Umsätze.

Der in diesem Tutorial verwendete Algorithmus ist [singuläre Spektralanalyse \(Singular Spectrum Analysis, SSA\)](#) . SSA zerlegt eine Zeitreihe in ihre Hauptkomponenten. Diese Komponenten können als Teile eines Signals interpretiert werden, das Trends, Rauschen, Saisonalität und vielen anderen Faktoren entspricht. Anschließend werden diese Komponenten rekonstruiert und verwendet, um Werte in der Zukunft vorherzusagen.

Erstellen einer Konsolenanwendung

1. Erstellen Sie eine neue C#-**Konsolenanwendung** mit dem Namen „BikeDemandForecasting“. Klicken Sie auf die Schaltfläche **Weiter**.
2. Wählen Sie .NET 6 als zu verwendendes Framework aus. Klicken Sie auf die Schaltfläche **Erstellen** .
3. Installieren des NuGet-Pakets **Microsoft.ML**

ⓘ Hinweis

In diesem Beispiel wird, sofern nicht anders angegeben, die neueste stabile Version der genannten NuGet-Pakete verwendet.

- a. Klicken Sie im Projektmappen-Explorer mit der rechten Maustaste auf das Projekt, und wählen Sie **NuGet-Pakete verwalten** aus.
- b. Wählen Sie „nuget.org“ als Paketquelle aus, wählen Sie die Registerkarte **Durchsuchen** aus, und suchen Sie nach **Microsoft.ML**.
- c. Aktivieren Sie das Kontrollkästchen **Vorabversion einbeziehen**.
- d. Wählen Sie die Schaltfläche **Installieren** aus.
- e. Wählen Sie die Schaltfläche **OK** im Dialogfeld **Vorschau der Änderungen** und dann die Schaltfläche **Ich stimme zu** im Dialogfeld „Zustimmung zur Lizenz“ aus, wenn Sie den Lizenzbedingungen für die aufgelisteten Pakete zustimmen.
- f. Wiederholen Sie diese Schritte für **System.Data.SqlClient** und **Microsoft.ML.TimeSeries**.

Vorbereiten und Verstehen der Daten

1. Erstellen Sie ein Verzeichnis namens *Data*.
2. Laden Sie die Datenbankdatei *DailyDemand.mdf* herunter, und speichern Sie sie im Verzeichnis *Data*.

! Hinweis

Die in diesem Tutorial verwendeten Daten stammen aus dem [UCI Bike Sharing-Dataset](#) . Fanaee-T, Hadi und Gama, Joao, „Event labeling combining ensemble detectors and background knowledge“, Progress in Artificial Intelligence (2013): S. 1-15, Springer Berlin Heidelberg, [Weblink](#) .

Das ursprüngliche Dataset enthält mehrere Spalten, die der Saisonalität und dem Wetter entsprechen. Aus Gründen der Kürze und weil der in diesem Tutorial verwendete Algorithmus nur die Werte einer einzigen numerischen Spalte benötigt, wurde das ursprüngliche Dataset so zusammengefasst, dass es nur die folgenden Spalten enthält:

- **dteday**: Das Datum der Beobachtung.
- **year**: Das codierte Jahr der Beobachtung (0 = 2011, 1 = 2012).
- **cnt**: Die Gesamtzahl der Fahrradvermietungen für diesen Tag.

Das ursprüngliche Dataset wird einer Datenbanktabelle mit dem folgenden Schema in einer SQL Server Datenbank zugeordnet.

SQL

```
CREATE TABLE [Rentals] (  
    [RentalDate] DATE NOT NULL,  
    [Year] INT NOT NULL,  
    [TotalRentals] INT NOT NULL  
);
```

Im Folgenden finden Sie ein Beispiel für die Daten:

 Tabelle erweitern

RentalDate	Jahr	TotalRentals
1/1/2011	0	985
1/2/2011	0	801
1/3/2011	0	1349

Erstellen von Eingabe- und Ausgabeklassen

1. Ersetzen Sie in *Program.cs* die vorhandenen `using`-Anweisungen durch Folgendes:

C#

```
using Microsoft.ML;  
using Microsoft.ML.Data;  
using Microsoft.ML.Transforms.TimeSeries;  
using System.Data.SqlClient;
```

2. Erstellen Sie die `ModelInput`-Klasse. Fügen Sie unter der `Program`-Klasse den folgenden Code hinzu.

C#

```
public class ModelInput  
{  
    public DateTime RentalDate { get; set; }  
  
    public float Year { get; set; }  
  
    public float TotalRentals { get; set; }  
}
```

Die `ModelInput`-Klasse enthält die folgenden Spalten:

- **RentalDate:** Das Datum der Beobachtung.
- **Year:** Das codierte Jahr der Beobachtung (0 = 2011, 1 = 2012).
- **TotalRentals:** Die Gesamtzahl der Fahrradvermietungen für diesen Tag.

3. Erstellen Sie die `ModelOutput`-Klasse unterhalb der neu erstellten `ModelInput`-Klasse.

C#

```
public class ModelOutput  
{  
    public float[] ForecastedRentals { get; set; }  
  
    public float[] LowerBoundRentals { get; set; }  
  
    public float[] UpperBoundRentals { get; set; }  
}
```

Die `ModelOutput`-Klasse enthält die folgenden Spalten:

- **ForecastedRentals:** Die vorhergesagten Werte für den Vorhersagezeitraum.

- **LowerBoundRentals:** Die vorhergesagten Mindestwerte für den Vorhersagezeitraum.
- **UpperBoundRentals:** Die vorhergesagten Maximalwerte für den Vorhersagezeitraum.

Definieren von Pfaden und Initialisieren von Variablen

1. Definieren Sie unterhalb der using-Anweisungen Variablen, um den Speicherort der Daten, die Verbindungszeichenfolge und den Speicherort für das trainierte Modell zu speichern.

C#

```
string rootDir = Path.GetFullPath(Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "../../../"));
string dbFilePath = Path.Combine(rootDir, "Data", "DailyDemand.mdf");
string modelPath = Path.Combine(rootDir, "MLModel.zip");
var connectionString = $"Data Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename={dbFilePath};Integrated Security=True;Connect Timeout=30;";
```

2. Initialisieren Sie die `mlContext`-Variable mit einer neuen Instanz von [MLContext](#), indem Sie die folgende Zeile hinzufügen, nachdem Sie die Pfade definiert haben.

C#

```
MLContext mlContext = new MLContext();
```

Die [MLContext](#)-Klasse ist der Ausgangspunkt für alle ML.NET-Vorgänge. Durch das Initialisieren von `mlContext` wird eine neue ML.NET-Umgebung erstellt, die für mehrere Objekte des Modellerstellungsworkflows verwendet werden kann. Die Klasse ähnelt dem Konzept von `DbContext` in Entity Framework.

Laden der Daten

1. Erstellen Sie das `DatabaseLoader`-Element, mit dem Datensätze des Typs `ModelInput` geladen werden.

C#

```
DatabaseLoader loader = mlContext.Data.CreateDatabaseLoader<ModelInput>();
```

2. Definieren Sie die Abfrage, um die Daten aus der Datenbank zu laden.

C#

```
string query = "SELECT RentalDate, CAST(Year as REAL) as Year,  
CAST(TotalRentals as REAL) as TotalRentals FROM Rentals";
```

ML.NET-Algorithmen erwarten, dass Daten vom Typ **Single** sind. Daher müssen numerische Werte aus der Datenbank, die nicht vom Typ **Real** (ein Gleitkommawert mit einfacher Genauigkeit) sind, in **Real** konvertiert werden.

Die Spalten `Year` und `TotalRental` sind Integertypen in der Datenbank. Mithilfe der integrierten Funktion `CAST` werden beide Werte in `Real` umgewandelt.

3. Erstellen Sie eine `DatabaseSource`, um eine Verbindung mit der Datenbank herzustellen und die Abfrage auszuführen.

C#

```
DatabaseSource dbSource = new DatabaseSource(SqlClientFactory.Instance,  
                                             connectionString,  
                                             query);
```

4. Laden Sie die Daten in eine `IDataView`.

C#

```
IDataView dataView = loader.Load(dbSource);
```

5. Das Dataset enthält Daten aus zwei Jahren. Nur Daten aus dem ersten Jahr werden für das Training verwendet, das zweite Jahr wird zum Vergleichen der tatsächlichen Werte mit der vom Modell generierten Vorhersage verwendet. Filtern Sie die Daten mithilfe der **FilterRowsByColumn**-Transformation.

C#

```
IDataView firstYearData = mlContext.Data.FilterRowsByColumn(dataView,  
"Year", upperBound: 1);  
IDataView secondYearData = mlContext.Data.FilterRowsByColumn(dataView,  
"Year", lowerBound: 1);
```

Für das erste Jahr werden nur die Werte in der Spalte `Year`, die kleiner als 1 ist, durch Festlegen des `upperBound`-Parameters auf 1 ausgewählt. Umgekehrt werden für das

zweite Jahr Werte größer oder gleich 1 ausgewählt, indem der `lowerBound` -Parameter auf 1 festgelegt wird.

Definieren einer Zeitreihenanalyse-Pipeline

1. Definieren Sie eine Pipeline, die `SsaForecastingEstimator` verwendet, um Werte in einem Zeitreihendataset vorherzusagen.

C#

```
var forecastingPipeline = mlContext.Forecasting.ForecastBySsa(  
    outputColumnName: "ForecastedRentals",  
    inputColumnName: "TotalRentals",  
    windowSize: 7,  
    seriesLength: 30,  
    trainSize: 365,  
    horizon: 7,  
    confidenceLevel: 0.95f,  
    confidenceLowerBoundColumn: "LowerBoundRentals",  
    confidenceUpperBoundColumn: "UpperBoundRentals");
```

Die `forecastingPipeline` nimmt für das erste Jahr 365 Datenpunkte an und teilt ein Zeitreihendataset in Stichproben von jeweils 30 Tagen (monatlich) auf, wie vom `seriesLength` -Parameter angegeben. Jede dieser Stichproben wird anhand eines wöchentlichen oder ein 7-tägigen Fensters analysiert. Bei der Ermittlung des prognostizierten Werts für die nächste(n) Zeitspanne(n) werden die Werte der letzten sieben Tage verwendet, um eine Prognose zu erstellen. Das Modell wird so festgelegt, dass sieben Zeitspannen in der Zukunft vorhergesagt werden, wie durch den `horizon` -Parameter definiert. Da eine Vorhersage eine fundierte Vermutung ist, ist sie nicht immer zu 100 % genau. Daher ist es gut, den Wertebereich in den besten und schlechtesten Szenarien zu kennen, wie durch die oberen und unteren Grenzen definiert. In diesem Fall wird der Vertrauensgrad für die untere und obere Grenze auf 95 % festgelegt. Der Vertrauensgrad kann entsprechend angehoben oder gesenkt werden. Je höher der Wert, desto größer ist der Bereich zwischen der oberen und unteren Grenze, um das gewünschte Maß an Vertrauen zu erreichen.

2. Verwenden Sie die `Fit`-Methode, um das Modell zu trainieren und die Daten an die zuvor definierte `forecastingPipeline` anzupassen.

C#

```
SsaForecastingTransformer forecaster =  
    forecastingPipeline.Fit(firstYearData);
```

Evaluieren des Modells

Bewerten Sie die Leistungsfähigkeit des Modells, indem Sie die Daten des nächsten Jahrs prognostizieren und mit den tatsächlichen Werten vergleichen.

1. Erstellen Sie am Ende der Datei **Program.cs** eine neue Hilfsmethode namens `Evaluate`.

```
C#  
  
Evaluate(IDataView testData, ITransformer model, MLContext mlContext)  
{  
  
}
```

2. Prognostizieren Sie in der `Evaluate`-Methode die Daten des zweiten Jahrs mithilfe der [Transform](#)-Methode mit dem trainierten Modell.

```
C#  
  
IDataView predictions = model.Transform(testData);
```

3. Verwenden Sie die [CreateEnumerable](#)-Methode, um die tatsächlichen Werte aus den Daten abzurufen.

```
C#  
  
IEnumerable<float> actual =  
    mlContext.Data.CreateEnumerable<ModelInput>(testData, true)  
        .Select(observed => observed.TotalRentals);
```

4. Verwenden Sie die [CreateEnumerable](#)-Methode, um die Vorhersagewerte abzurufen.

```
C#  
  
IEnumerable<float> forecast =  
    mlContext.Data.CreateEnumerable<ModelOutput>(predictions, true)  
        .Select(prediction => prediction.ForecastedRentals[0]);
```

5. Berechnen Sie die Differenz zwischen den tatsächlichen Werten und den Vorhersagewerten, die im Allgemeinen als „Fehler“ bezeichnet wird.

```
C#  
  
var metrics = actual.Zip(forecast, (actualValue, forecastValue) =>
```

```
actualValue - forecastValue);
```

6. Messen Sie die Leistung, indem Sie die Werte Mean Absolute Error (Mittlerer absoluter Fehler) und Root Mean Squared Error (Mittlerer quadratischer Fehler) berechnen.

C#

```
var MAE = metrics.Average(error => Math.Abs(error)); // Mean Absolute Error
var RMSE = Math.Sqrt(metrics.Average(error => Math.Pow(error, 2))); // Root Mean Squared Error
```

Die folgenden Metriken werden verwendet, um die Leistung auszuwerten:

- **Mean Absolute Error** (Mittlerer absoluter Fehler): Misst, wie nahe Vorhersagen am tatsächlichen Wert liegen. Dieser Wert liegt zwischen 0 und unendlich. Je näher der Wert an 0 ist, desto besser ist die Qualität des Modells.
- **Root Mean Squared Error** (Mittlerer quadratischer Fehler): Fasst den Fehler im Modell zusammen. Dieser Wert liegt zwischen 0 und unendlich. Je näher der Wert an 0 ist, desto besser ist die Qualität des Modells.

7. Geben Sie die Metriken an die Konsole aus.

C#

```
Console.WriteLine("Evaluation Metrics");
Console.WriteLine("-----");
Console.WriteLine($"Mean Absolute Error: {MAE:F3}");
Console.WriteLine($"Root Mean Squared Error: {RMSE:F3}\n");
```

8. Rufen Sie die `Evaluate`-Methode unter der `Fit()`-Methode auf.

C#

```
Evaluate(secondYearData, forecaster, mlContext);
```

Speichern des Modells

Wenn Sie mit Ihrem Modell zufrieden sind, speichern Sie es zur späteren Verwendung in anderen Anwendungen.

1. Erstellen Sie unter der `Evaluate()`-Methode ein [TimeSeriesPredictionEngine](#)-Element. [TimeSeriesPredictionEngine](#) ist eine bequeme Methode, um einzelne Vorhersagen zu treffen.

C#

```
var forecastEngine = forecaster.CreateTimeSeriesEngine<ModelInput,
ModelOutput>(mlContext);
```

- Speichern Sie das Modell in einer Datei namens `MLModel.zip`, wie durch die zuvor definierten `modelPath`-Variable angegeben. Verwenden Sie die [Checkpoint](#)-Methode, um das Modell zu speichern.

C#

```
forecastEngine.CheckPoint(mlContext, modelPath);
```

Verwenden des Modells zum Vorhersagen des Bedarfs

- Erstellen Sie unter der `Evaluate`-Methode eine neue Hilfsmethode namens `Forecast`.

C#

```
void Forecast(IDataView testData, int horizon,
TimeSeriesPredictionEngine<ModelInput, ModelOutput> forecaster,
MLContext mlContext)
{
}
}
```

- Verwenden Sie innerhalb der `Forecast`-Methode die [Predict](#)-Methode, um die Vermietungen der nächsten sieben Tage vorherzusagen.

C#

```
ModelOutput forecast = forecaster.Predict();
```

- Bringen Sie die tatsächlichen und die vorhergesagten Werte für sieben Zeiträume in Einklang.

C#

```
IEnumerable<string> forecastOutput =
    mlContext.Data.CreateEnumerable<ModelInput>(testData,
reuseRowObject: false)
    .Take(horizon)
    .Select((ModelInput rental, int index) =>
```

```
{
    string rentalDate = rental.RentalDate.ToShortDateString();
    float actualRentals = rental.TotalRentals;
    float lowerEstimate = Math.Max(0, forecast.LowerBoundRentals[index]);
    float estimate = forecast.ForecastedRentals[index];
    float upperEstimate = forecast.UpperBoundRentals[index];
    return $"Date: {rentalDate}\n" +
        $"Actual Rentals: {actualRentals}\n" +
        $"Lower Estimate: {lowerEstimate}\n" +
        $"Forecast: {estimate}\n" +
        $"Upper Estimate: {upperEstimate}\n";
});
```

4. Iterieren Sie durch die Vorhersageausgabe, und zeigen Sie diese in der Konsole an.

C#

```
Console.WriteLine("Rental Forecast");
Console.WriteLine("-----");
foreach (var prediction in forecastOutput)
{
    Console.WriteLine(prediction);
}
```

Ausführen der Anwendung

1. Rufen Sie unter dem `Checkpoint()`-Methodenaufruf die `Forecast`-Methode auf.

C#

```
Forecast(secondYearData, 7, forecastEngine, mlContext);
```

2. Führen Sie die Anwendung aus. Eine ähnliche Ausgabe wie die folgende sollte in der Konsole angezeigt werden. Aus Gründen der Kürze wurde die Ausgabe komprimiert.

text

```
Evaluation Metrics
-----
Mean Absolute Error: 726.416
Root Mean Squared Error: 987.658

Rental Forecast
-----
Date: 1/1/2012
Actual Rentals: 2294
Lower Estimate: 1197.842
```

Forecast: 2334.443
Upper Estimate: 3471.044

Date: 1/2/2012
Actual Rentals: 1951
Lower Estimate: 1148.412
Forecast: 2360.861
Upper Estimate: 3573.309

Bei der Untersuchung der tatsächlichen und der vorhergesagten Werte werden die folgenden Beziehungen angezeigt:

Bike Rentals(January 1 - January 7)



Obwohl die vorhergesagten Werte nicht die genaue Anzahl von Vermietungen vorhersagen, stellen sie einen engeren Bereich von Werten bereit, der es im Betrieb ermöglicht, die Verwendung von Ressourcen zu optimieren.

Herzlichen Glückwunsch! Sie haben nun erfolgreich ein Machine Learning-Zeitreihenmodell erstellt, um den Bedarf für die Fahrradvermietung zu prognostizieren.

Sie finden den Quellcode für dieses Tutorial im Repository [dotnet/machinelearning-samples](#) .

Nächste Schritte

- [Machine Learning-Aufgaben in ML.NET](#)
- [Verbessern der Modellgenauigkeit](#)

Last updated on 09.05.2023